



CLOUDPANDITH ONLINE TRAININGS



Master Data Engineering with Expert-Led Live Training

◇ Azure Data Factory

Complete ETL/ELT

◇ Azure Databricks

Big Data & PySpark

◇ Microsoft Fabric

End-to-End Analytics

📞 ENROLL: **8904424822** | 🌐 www.cloudpandith.com

✓ Live Sessions ✓ Real Projects ✓ Expert Trainers ✓ Interview Prep ✓ Lifetime Access

- ◆ Live Classes
- ◆ Real-Time Projects
- ◆ Interview Preparation
- ◆ Lifetime Access

Index

1. Current challenges with large-scale data processing
2. How Change Data Feed solves these challenges
3. What is Change Data Feed (CDF)
4. When to use CDF and when not to use it
5. Real-time and incremental processing use cases
6. Enabling CDF in Delta Lake
7. Reading changed records using SQL and PySpark
8. Building Audit Trail tables using CDF
9. Building Gold tables incrementally using CDF

Delta Lake Features: Optimize Command & Auto Compaction

Introduction

In Delta Lake, managing file sizes is very important for achieving better query performance and efficient storage utilization. During continuous data ingestion, streaming operations, or frequent updates, many small files are created. This situation is known as the **Small File Problem**.

To solve this issue, Delta Lake provides:

- **OPTIMIZE Command**
- **Auto Compaction Feature**

These features help improve query speed, reduce metadata overhead, and optimize storage management.

What is Small File Size Issue?

The small file problem occurs when a Delta table contains a very large number of tiny files instead of fewer larger files. This negatively affects performance and resource utilization.

Problems Caused by Small Files

1. Reduced Query Performance

- Query engines must read metadata from many files.
- More I/O operations increase latency.
- Distributed systems cannot efficiently utilize parallel processing.

2. Increased Storage Overhead

- Delta Lake tracks every file in the transaction log.
- Large metadata increases management complexity.
- Cloud storage systems may incur additional costs for handling many files.

3. Inefficient Data Skipping

- Predicate pushdown and partition pruning become less effective.
- More files need to be scanned during queries.

4. Increased Management Complexity

- More checkpoints and transaction logs are generated.
- Additional maintenance operations are required.

5. Resource Utilization Issues

- High network overhead due to multiple file reads.
- Inefficient memory and compute usage.

Layman Example

Imagine storing 10,000 photos individually in separate small envelopes instead of organizing them into a few large folders. Searching through thousands of envelopes takes much longer than accessing a few organized folders.

Similarly, Delta Lake performs better with fewer large files instead of many small files.

OPTIMIZE Command in Delta Lake

The OPTIMIZE command is used to compact many small files into fewer large files.

This improves:

- Query performance
- Read efficiency
- Storage management

Benefits of OPTIMIZE

Improved Query Performance

- Reduces the number of files scanned during query execution.

Efficient File Sizes

- Creates larger files that are easier to process.

Better Data Readability

- Optimized layout improves analytical operations.
-

Syntax of OPTIMIZE Command

Optimize a Path-Based Delta Table

```
OPTIMIZE '/path/to/delta/table';
```

Optimize a Registered Delta Table

```
OPTIMIZE delta_table_name;
```

Another Path-Based Syntax

```
OPTIMIZE delta.`/path/to/delta/table`;
```

Optimize Only Specific Partitions

```
OPTIMIZE delta_table_name  
WHERE date >= '2017-01-01';
```

This is useful when you want to optimize only recent data instead of the entire table.

Auto Compaction in Delta Lake

Auto Compaction automatically merges small files into larger files after write operations.

It helps reduce operational overhead because compaction happens automatically without manual intervention.

Key Benefits

Combining Small Files

Automatically merges small files within partitions.

Trigger Timing

Runs automatically after successful write operations.

Scope

Only uncompact files are processed.

Auto Compaction Configurations

Maximum File Size

You can configure output file size using:

```
spark.databricks.delta.autoCompact.maxFileSize
```

Minimum Number of Files

Auto compaction triggers only when a minimum number of small files exist.

Configuration:

spark.databricks.delta.autoCompact.minNumFiles

How to Enable Auto Compaction

Method 1: Table Property

```
ALTER TABLE table_name
SET TBLPROPERTIES (
  'delta.autoOptimize.autoCompact' = 'true'
);
```

Method 2: Spark Session Configuration

```
spark.conf.set(
  "spark.databricks.delta.autoCompact.enabled",
  "true"
)
```

Conclusion

The small file problem is one of the most common performance issues in Delta Lake. Delta Lake provides two important mechanisms to solve this:

1. **OPTIMIZE Command** — Manual compaction of small files.
2. **Auto Compaction** — Automatic background compaction after writes.

Using these features improves:

- Query performance
- Storage efficiency
- Resource utilization
- Metadata management

Proper file optimization is essential for building scalable and high-performance data engineering solutions in Databricks and Delta Lake.